



Formerly known as
Dhirubhai Ambani Institute of Information and Communication Technology

A Summer Internship Training Report

on

“Full Custom Chip Design of AIML Hardware”

Submitted in Partial Fulfillment of the Requirements for

3FT901CC24 – SUMMER INTERNSHIP

By

Vivek Rakeshsingh Singh (23BEI073)



**ELECTRONICS AND INSTRUMENTATION ENGINEERING
INSTITUTE OF TECHNOLOGY
NIRMA UNIVERSITY
AHMEDABAD – 382 481
JULY 2026**

Submitted by	Vivek Rakeshsingh Singh
Roll No.	23BEI073
Programme	B. Tech in Electronics & Instrumentation Engineering
Institution	Dhirubhai Ambani University
Department	VLSI and Embedded Research Group
External Guide	Prof. Tapas Kumar Maiti
Internship Duration	11/05/2026 to 10/07/2026 (9 Weeks)

- Certificate



Certificate ID: 2026SRI06

Declaration

I hereby declare that the work presented in this Summer Research Internship Report titled **“Custom CMOS Design of a Neural Network Accelerator with On-Chip Training Hardware in 90 nm Technology”** is the result of my own work carried out under the guidance of **Prof. Tapas Kumar Maiti**, DA-IICT, Gandhinagar, during the period of May – July 2026.

I further declare that:

- i. The work contained in this report has not been previously submitted for any degree or any other academic distinction anywhere.
- ii. All external sources of information have been appropriately cited and acknowledged in the references.
- iii. The Verilog RTL designs, CDL netlists, synthesis scripts, and layout work presented herein are original contributions.
- iv. The technical content related to the reference architecture has been duly cited and no plagiarism has been committed.

Vivek [Surname]

Roll No.: 23BEI073

B.Tech – EIE, Nirma University

Signature: _____

Date: _____

Acknowledgement

I take this opportunity to express my deepest and most sincere gratitude to all those who supported, guided, and inspired me throughout this Summer Research Internship.

First and foremost, I am profoundly grateful to **Prof. Tapas Kumar Maiti**, DA-IICT, Gandhinagar, for giving me the extraordinary opportunity to work under his mentorship. His depth of knowledge in VLSI design, his patience in explaining complex concepts, and his constant encouragement made this one of the most technically enriching experiences of my academic life. His insistence on understanding problems from first principles — rather than simply using black-box tools — has permanently changed how I approach engineering challenges.

I would like to thank the **DA-IICT Summer Research Internship (SRI) Programme** for providing a structured and academically rigorous platform for students to engage with cutting-edge research. The infrastructure access — particularly the Cadence EDA tools suite, the gpd090 PDK, and the VLSI laboratory — was invaluable.

I am grateful to the faculty and research staff of the **Department of Information and Communication Technology, DA-IICT**, for their support and for the stimulating intellectual environment they have fostered.

I express my sincere thanks to the professors and faculty of the **Department of Electronics and Instrumentation Engineering, Nirma University**, particularly my internal guide, for their continuous academic support and for encouraging me to pursue research opportunities.

I thank my fellow interns and colleagues at DA-IICT for the collaborative spirit, the technical discussions, and for making the experience personally memorable. The evenings spent debugging synthesis scripts and the collective frustration and satisfaction of seeing our first layouts pass DRC are memories I will carry forward.

Finally, and most importantly, I express my deepest gratitude to my **family** for their unconditional support, encouragement, and patience throughout my academic journey.

Vivek Singh

Ahmedabad, July 2026

Abstract

This report documents the work carried out during the Summer Research Internship (SRI) at the Dhirubhai Ambani Institute of Information and Communication Technology (DA-IICT), Gandhinagar, under the guidance of Prof. Tapas Kumar Maiti, during the period of May to July 2026.

The primary project undertaken during the internship was the complete custom CMOS design and physical layout of a hardware neural network (NN) accelerator chip in the gpdk090 90 nm process design kit. The accelerator implements both forward and backward propagation entirely in dedicated silicon datapaths, enabling on-chip training without any software intervention.

The forward propagation engine comprises parameterised multiply-accumulate (MAC) units operating in Q1.3 4-bit fixed-point arithmetic, a custom 16-entry NOR-based ROM implementing the sigmoid activation function, on-chip training data ROMs, and weight and bias register banks with configurable initial values. The backward propagation engine implements concurrent derivative-of-activation computation, layer-wise delta propagation, gradient multiplication, and sign-gradient weight update accumulators operating over a four-sample mini-batch training loop.

The entire standard cell library was designed from transistor level using gpdk090 SPICE device parameters, covering five primitive cells and seven derived wrapper cells. Logic synthesis was performed using Cadence Genus targeting a 100 MHz clock at the gpdk090 slow corner. The complete physical layout was implemented and verified in Cadence Virtuoso; DRC and LVS verification using Assura with the gpdk090 rule deck confirmed correctness of the implementation.

The complete design spans 22 Verilog RTL modules, approximately 4,080 leaf gate instances, and 401 flip-flop instances. The work demonstrates a complete RTL-to-layout ASIC flow for a custom AI hardware accelerator without reliance on any third-party standard cell library.

Keywords: Neural network accelerator, VLSI, custom CMOS, fixed-point arithmetic, hardware backpropagation, Cadence Genus, Cadence Virtuoso, gpdk090, sigmoid LUT ROM, physical design, DRC, LVS.

Contents

Declaration	1
Acknowledgement	2
Abstract	3
Contents	7
List of Figures	7
List of Tables	8
List of Abbreviations	9
1 Introduction	11
1.1 Background and Motivation	11
1.2 About DA-IICT	11
1.3 About the Summer Research Internship Programme	12
1.4 Objectives of the Internship	12
1.5 Overview of the Report	13
2 Background and Literature Review	14
2.1 Hardware Neural Network Accelerators	14
2.2 Fixed-Point Arithmetic in Neural Networks	14
2.3 Activation Function Hardware	15
2.4 Hardware Backpropagation	15
2.5 Custom CMOS Design and NOR ROM Architecture	16
3 Tools and Technologies	17
3.1 Electronic Design Automation (EDA) Suite	17
3.1.1 Cadence Genus Synthesis Solution	17
3.1.2 Cadence Virtuoso	17
3.1.3 Icarus Verilog	18
3.1.4 GTKWave	18

3.2	Process Design Kit (PDK)	18
3.2.1	gpdk090 – Generic 90 nm PDK	18
3.3	Programming Languages and Scripting	19
3.3.1	Verilog HDL	19
3.3.2	TCL Scripting	19
3.3.3	SPICE / CDL Netlist Format	19
3.3.4	LaTeX	19
4	Week-by-Week Internship Activities	20
4.1	Overview	20
4.2	Week 1: Orientation, Literature Review, and Environment Setup	20
4.2.1	Orientation and Project Scoping	20
4.2.2	Literature Review	20
4.2.3	Environment Setup	21
4.3	Week 2: Architecture Design and RTL Development – Forward Pass	21
4.3.1	Architecture Planning	21
4.3.2	Verilog RTL – Forward Pass Modules	22
4.3.3	Q1.3 Fixed-Point MAC Design	22
4.4	Week 3: RTL Development – Backward Pass and Debugging	23
4.4.1	Backward Pass RTL Modules	23
4.4.2	Critical Bug: Unsigned Address in mem4	23
4.4.3	Critical Bug: Wrong Bit Extraction in dadz4	24
4.4.4	Critical Bug: Combinational/Registered Race in z2_4	24
4.5	Week 4: Genus Synthesis and Standard Cell Library Design	25
4.5.1	Genus Synthesis Flow	25
4.5.2	Standard Cell Library Design	25
4.6	Week 5: Custom ROM Design and Hierarchical CDL	27
4.6.1	mem4 NOR ROM Design	27
4.6.2	ROM Array Programming	27
4.6.3	CDL File Structure	28
4.7	Week 6: Physical Design – Standard Cells and Forward4 Layout	28
4.7.1	Cell Layout Methodology	28
4.7.2	forward4 Layout	28
4.8	Week 7: Physical Design – Backward4 and nn_core4 Layout	29
4.8.1	backward4 CDL Import and Layout	29
4.8.2	nn_core4 Top-Level Assembly	30
4.9	Week 8: Verification, Documentation, and Reporting	30
4.9.1	DRC Verification	30
4.9.2	LVS Verification	30

4.9.3	Report and Paper Writing	31
5	System Architecture	32
5.1	Network Topology	32
5.2	Data Representation	32
5.3	Initial Weight Configuration	32
5.4	Pipeline Timing	32
6	Forward Propagation Engine	34
6.1	MAC Unit Design	34
6.1.1	Arithmetic	34
6.1.2	Implementation Complexity	34
6.2	Sigmoid ROM (mem4)	34
6.3	Weight and Bias Registers	35
6.3.1	Weight Register Variants	35
6.3.2	gen_din_sel4 Control Block	35
7	Backward Propagation Engine	36
7.1	Mathematical Formulation	36
7.2	Derivative of Activation (dadz4)	36
7.3	Gradient Accumulation and Sign-Gradient Update	37
8	Custom Cell Library and mem4 ROM	38
8.1	Standard Cell Library Overview	38
8.2	XNOR2X1 Design	38
8.3	mem4 NOR ROM Hierarchy	39
9	Physical Design and Verification	40
9.1	CDL Import Flow	40
9.1.1	Import Settings	40
9.2	Layout Generation	40
9.2.1	Standard Cell Layout	40
9.2.2	Wrapper Cell Layout	41
9.3	DRC Results	41
9.4	LVS Results	41
10	Results and Analysis	43
10.1	Synthesis Results	43
10.2	Design Complexity	43
10.3	Comparison with Reference Architecture	43
10.4	Arithmetic Accuracy Analysis	43

11 Skills Acquired and Learning Outcomes	45
11.1 Technical Skills	45
11.1.1 VLSI Design Flow Proficiency	45
11.1.2 Digital Arithmetic and Hardware Design	45
11.1.3 AI Hardware Architecture	46
11.2 Professional Skills	46
11.3 Personal Growth	46
12 Challenges and Solutions	48
12.1 Challenges Encountered	48
13 Conclusion and Future Work	50
13.1 Conclusion	50
13.2 Future Work	50
A Complete RTL Module List	54
B CDL File Summary	55
C Internship Photos	56

List of Figures

4.1 Physical layout of the <code>forward4</code> forward propagation engine.	29
4.2 Physical layout of the <code>backward4</code> backward propagation engine. The layout shows <code>dadz</code> units, delta computation blocks, gradient multipliers, accumulator chains, and pipeline delay registers.	30
4.3 Full chip physical layout of <code>nn_core4</code> . This layout integrates <code>forward4</code> , <code>backward4</code> , <code>gen_din_sel4</code> , pipeline delay chains, and top-level control logic.	30
C.1 With faculty mentor Prof. Tapas Kumar Maiti at DA-IICT, Gandhinagar. Photo taken at the DA-IICT VLSI laboratory, June 2026.	56
C.2 Working at the Cadence EDA workstation in the DA-IICT VLSI Lab, showing the Cadence Virtuoso Layout Editor with the <code>nn_core4</code> physical layout.	57

List of Tables

3.1	gpd090 Key Process Parameters	19
4.1	Forward Pass RTL Modules	22
4.2	Backward Pass RTL Modules	23
4.3	Sigmoid LUT Content for mem4 (Q1.3 addresses and outputs)	27
4.4	ROM Array Programming Matrix (N = NMOS present = bit 0)	27
4.5	Common DRC Violations and Fixes	31
5.1	Q1.3 Fixed-Point Value Mapping	33
5.2	Initial Weight and Bias Values (Q1.3 Integer and Real)	33
5.3	Pipeline Stage Timing	33
8.1	Complete Custom Standard Cell Library	38
9.1	DRC Violation Summary During Development	41
10.1	Logic Synthesis Results – Cadence Genus, gpd090 Slow Corner	43
10.2	Design Complexity by Block	43
10.3	Comparison with Reference Architecture [8]	44
A.1	Complete List of RTL Verilog Modules	54
B.1	CDL Files Generated During the Internship	55

List of Abbreviations

Abbreviation	Full Form
ANN	Artificial Neural Network
ASIC	Application-Specific Integrated Circuit
CDL	Circuit Description Language
CMOS	Complementary Metal-Oxide Semiconductor
CTS	Clock Tree Synthesis
DA-IICT	Dhirubhai Ambani Institute of Information and Communication Technology
DFF	D Flip-Flop
DRC	Design Rule Check
EDA	Electronic Design Automation
EIE	Electronics and Instrumentation Engineering
FBD	Function Block Diagram
FF	Flip-Flop
GDSII	Graphic Design System II
gpd090	Generic Process Design Kit, 90 nm
HDL	Hardware Description Language
IC	Integrated Circuit
LUT	Lookup Table
LVS	Layout Versus Schematic
MAC	Multiply-Accumulate
MLP	Multilayer Perceptron
MOS	Metal-Oxide Semiconductor
NMOS	N-type Metal-Oxide Semiconductor
PDK	Process Design Kit
PMOS	P-type Metal-Oxide Semiconductor
Q1.3	Fixed-point format: 1 sign bit, 0 integer bits, 3 fractional bits
ROM	Read-Only Memory
RTL	Register-Transfer Level
SDC	Synopsys Design Constraints
SPICE	Simulation Program with Integrated Circuit Emphasis
SRI	Summer Research Internship
STA	Static Timing Analysis

TG	Transmission Gate
VLSI	Very Large Scale Integration
WNS	Worst Negative Slack

Chapter 1

Introduction

1.1 Background and Motivation

The rapid advancement of artificial intelligence (AI) and machine learning (ML) in the past decade has fundamentally transformed the landscape of computing. Neural networks, which underpin modern AI — from image recognition systems to large language models — demand enormous computational resources. This has driven an unprecedented demand for dedicated hardware accelerators: custom silicon chips designed specifically to accelerate the core mathematical operations of neural networks.

While frameworks like TensorFlow and PyTorch have democratised the use of neural networks, they rely on general-purpose hardware (CPUs, GPUs) that executes neural network operations as software. A dedicated hardware accelerator, by contrast, maps these operations directly into fixed silicon logic, achieving orders-of-magnitude improvements in energy efficiency and throughput. This is the principle behind chips like Google’s Tensor Processing Unit (TPU), Apple’s Neural Engine, and NVIDIA’s Tensor Core architecture.

What makes hardware neural network design truly fascinating — and genuinely challenging — is the depth of the design hierarchy one must traverse: from the high-level mathematical description of a neural network, through algorithmic choices about fixed-point precision, through Register-Transfer Level (RTL) Verilog design, through logic synthesis, down to the transistor-level physical layout of every gate and memory cell in the chip. At each level, design decisions made above constrain possibilities below, and constraints from the physical level propagate upward as limitations on what algorithms can feasibly be implemented.

This internship project was motivated by the desire to traverse this entire hierarchy — not just to understand individual levels in isolation, but to build a complete, verified ASIC implementation of a neural network accelerator, understanding how every abstraction layer connects to the one below it.

1.2 About DA-IICT

The Dhirubhai Ambani Institute of Information and Communication Technology (DA-IICT) is a premier private university located in Gandhinagar, Gujarat, India. Established

in 2001 by the Reliance Foundation in memory of the visionary industrialist Dhirubhai Ambani, DA-IICT has established itself as one of India's leading institutions in the fields of information technology, communication technology, and electronics.

The institute offers undergraduate, postgraduate, and doctoral programmes in ICT and related fields. Its faculty includes accomplished researchers with international recognition in areas including VLSI design, signal processing, wireless communications, machine learning, and embedded systems. The institute's research infrastructure includes well-equipped electronics and VLSI laboratories with access to industry-standard EDA (Electronic Design Automation) tools, including the Cadence design suite used in this project.

The VLSI research group at DA-IICT, led by faculty members with expertise in digital IC design, has been involved in research on low-power design, hardware accelerators for AI/ML workloads, and custom CMOS design. This internship was undertaken under the direct supervision of **Prof. Tapas Kumar Maiti**, whose research interests lie at the intersection of digital VLSI design, hardware implementation of machine learning algorithms, and computer arithmetic.

1.3 About the Summer Research Internship Programme

The DA-IICT Summer Research Internship (SRI) Programme provides B.Tech and M.Tech students from across India with the opportunity to engage in research-level work under the guidance of DA-IICT faculty. The programme typically runs during the summer vacation period (May to July) and involves students working on defined research projects in close collaboration with their faculty mentors.

The SRI programme is distinguished from industry internships in its emphasis on research methodology, theoretical grounding, and original technical contribution. Students are expected not merely to learn existing tools, but to apply them to open-ended problems and produce technically sound, potentially publishable work. At the conclusion of the programme, students submit a formal technical report in IEEE format.

This internship was undertaken as a project in the domain of VLSI design for AI hardware acceleration, spanning the complete chip design flow from RTL to physical layout.

1.4 Objectives of the Internship

The primary objectives of this internship were:

O1. Understand AI hardware architecture: Study the mathematical foundations of

neural network forward and backward propagation and translate them into hardware-implementable architectures.

- O2. RTL design and simulation:** Design, write, and simulate a complete Verilog RTL implementation of a neural network accelerator incorporating both forward and backward propagation.
- O3. Fixed-point arithmetic:** Implement and verify correct fixed-point arithmetic (Q1.3 format) for all MAC operations, gradient computation, and weight update logic.
- O4. Logic synthesis:** Learn and execute the complete Cadence Genus synthesis flow, including constraint development, optimisation, and timing closure at 100 MHz in gpd090 90 nm technology.
- O5. Custom cell design:** Design a complete standard cell library from transistor level using gpd090 SPICE models, including combinational and sequential cells.
- O6. Custom ROM design:** Design and implement a custom 16×4-bit NOR ROM for sigmoid activation function lookup, including full transistor-level CDL and layout.
- O7. Physical design:** Execute the complete hierarchical CDL import, schematic generation, and physical layout flow in Cadence Virtuoso.
- O8. Design verification:** Perform DRC and LVS verification of all custom cells and the complete chip using Assura with the gpd090 rule deck.

1.5 Overview of the Report

This report is organised as follows. Chapter 2 reviews the relevant background literature on hardware neural network accelerators, fixed-point arithmetic, and activation function hardware. Chapter 3 describes the tools, software, and hardware infrastructure used during the internship. Chapter 4 provides a week-by-week chronological account of the internship activities. Chapter 5 describes the overall system architecture of the implemented chip. Chapter 6 details the forward propagation engine. Chapter 7 details the backward propagation engine. Chapter 8 describes the custom standard cell library and the mem4 sigmoid ROM. Chapter 9 covers the physical design and verification flow. Chapter 10 presents implementation results and a comparison with related work. Chapter 11 discusses skills acquired and personal growth. Chapter 12 addresses challenges encountered and their solutions. Chapter 13 concludes the report. References and appendices follow.

Chapter 2

Background and Literature Review

2.1 Hardware Neural Network Accelerators

The earliest hardware implementations of artificial neural networks date to the 1980s and 1990s, when analog VLSI circuits were used to directly implement the sigmoid function and weighted summation in continuous-time domains [1]. The rise of digital VLSI enabled more precise, programmable, and scalable implementations.

Modern hardware ANN accelerators broadly fall into two categories: those based on programmable platforms (FPGAs) and those implemented as fixed-function ASICs. FPGA-based implementations offer flexibility and rapid prototyping but are limited in energy efficiency relative to custom silicon. ASIC implementations, exemplified by Google’s TPU [2] and NVIDIA’s Tensor Cores, achieve the highest energy efficiency by dedicating every transistor to a specific function.

The systolic array, used in the TPU, is a particularly elegant architecture for matrix multiplication: data flows through a two-dimensional array of multiply-accumulate units in a pipelined fashion, such that each unit passes its partial result to its neighbour with minimal global data movement. Eyeriss [3] explored the concept of dataflow-based architectures, demonstrating that the choice of how to schedule data movement between processing elements and memory hierarchies can have as large an impact on energy efficiency as the arithmetic units themselves.

The common theme across these designs is that efficient hardware neural network acceleration requires co-designing the arithmetic precision, the memory hierarchy, and the control logic together. This holistic design philosophy guided the architecture choices in this internship project.

2.2 Fixed-Point Arithmetic in Neural Networks

Real-valued neural networks are conventionally implemented using 32-bit floating-point arithmetic in software. In hardware, floating-point arithmetic requires substantial logic area and has relatively high energy consumption per operation compared to fixed-point equivalents. For hardware acceleration, fixed-point arithmetic is strongly preferred.

Fixed-point representations sacrifice dynamic range for simplicity: the decimal point is placed at a fixed position within the bit word. A $Q_{m.n}$ format has m integer bits and n

fractional bits, covering a range of approximately $[-2^m, 2^m - 2^{-n}]$ with resolution 2^{-n} .

The choice of fixed-point format is a critical design decision. Too few fractional bits cause quantisation noise that impairs network accuracy; too many bits increase hardware cost. For simple binary classification tasks, Q1.3 4-bit precision is sufficient when combined with appropriate training algorithms. Gupta et al. [5] demonstrated that stochastic rounding can enable training convergence even with very limited precision.

In this project, Q1.3 4-bit arithmetic is used throughout: 1 sign bit and 3 fractional bits, covering $[-1.0, +0.875]$ with resolution 0.125. The multiply-accumulate operation on two Q1.3 values produces an 8-bit result, from which bits [6:3] are extracted to recover the correctly scaled Q1.3 product.

2.3 Activation Function Hardware

The sigmoid function $\sigma(x) = 1/(1+e^{-x})$ cannot be computed directly in digital hardware without significant area and latency cost. Several approximation strategies have been proposed:

Piecewise linear (PWL) approximation [7] divides the sigmoid domain into linear segments, each described by a slope and intercept stored in a table. This allows the sigmoid to be computed with adders and multipliers rather than exponential units, but requires careful region boundary selection.

Lookup table (LUT) methods directly store sigmoid output values indexed by input. The required table size grows exponentially with bit width: a 16-bit input sigmoid requires $2^{16} = 65,536$ entries. For our 4-bit Q1.3 input, only 16 entries are needed, making a direct LUT ROM practical.

Hybrid methods [8] combine region-based decomposition with LUT storage, using a decoder to select between constant regions (saturation), a linear approximation, and a small LUT for the nonlinear central region. This reduces storage requirements while maintaining accuracy for higher bit widths.

This project uses a direct 16-entry LUT ROM approach, which is optimal for the 4-bit Q1.3 domain. The ROM is implemented as a custom pseudo-NMOS NOR ROM rather than using standard logic cells, enabling compact silicon area.

2.4 Hardware Backpropagation

Hardware implementation of the backpropagation training algorithm is significantly more challenging than forward inference because it requires three sequential operations: forward pass (compute activations), backward pass (compute deltas and gradients), and weight update (modify stored weights). These operations have data dependencies that impose strict ordering constraints.

Relatively few academic works implement hardware training in silicon. Torabi et al. [9] implemented a hardware backpropagation unit targeting an FPGA using 8-bit fixed-point arithmetic, demonstrating convergence on simple classification tasks. The key challenge is that the weight update step involves very small numbers (gradients are products of three small values), which underflow in low-precision fixed-point arithmetic.

The sign-gradient descent approach used in this project addresses this precision problem by making the weight update binary: the sign of the accumulated gradient determines whether to add or subtract 1 LSB from the weight, regardless of gradient magnitude. This is equivalent to a stochastic subgradient descent approach and converges correctly for simple classification tasks.

2.5 Custom CMOS Design and NOR ROM Architecture

A NOR ROM is one of the simplest and most area-efficient structures for implementing a small read-only memory in custom CMOS [10]. The architecture consists of a decoder (producing wordline signals), a bitcell array (where NMOS transistors at programmed positions pull bitlines low when their wordline is asserted), and PMOS pull-up devices (maintaining bitlines high when no NMOS cell is active).

The NOR ROM architecture is preferred over NAND ROMs for small memories because individual wordlines drive exactly the cells of one row, giving a simpler decoder and lower power. For our 16-entry $\times$ 4-bit sigmoid ROM, the NOR ROM requires:

- 4 inverters (address complement generation)
- $16 \times \text{NAND4} + 16 \times \text{INV}$ for the 4-to-16 decoder (total 192T)
- 3 PMOS pull-up devices (one per active bitline)
- Up to 23 NMOS bitcell transistors (programmed positions only)
- Output latches ($3 \times \text{D-latch}$)

The custom ROM macro `mem4` encapsulates this architecture as a hierarchical black box in the synthesis flow, preventing synthesis tools from attempting to re-implement its function in standard cells.

Chapter 3

Tools and Technologies

3.1 Electronic Design Automation (EDA) Suite

The primary EDA tools used during this internship were from the Cadence Design Systems suite, accessed through the DA-IICT VLSI laboratory. These are industry-standard tools used by leading semiconductor companies worldwide.

3.1.1 Cadence Genus Synthesis Solution

Cadence Genus is a logic synthesis tool that translates HDL RTL descriptions into technology-mapped gate-level netlists optimised for a specific process technology. Genus operates in multiple phases:

1. **Elaboration:** Parsing and elaborating the HDL, constructing an internal design representation.
2. **Generic synthesis (`syn_gen`):** Technology-independent optimisation, including constant propagation, dead code elimination, and Boolean simplification.
3. **Mapping (`syn_map`):** Technology mapping – selecting gates from the target library to implement each Boolean function.
4. **Optimisation (`syn_opt`):** Timing closure, area reduction, and power optimisation.

Synthesis is guided by **Synopsys Design Constraints (SDC)** files that specify clock period, input/output delays, false paths, and multi-cycle paths. Critical to our flow was the requirement that the `create_constraint_mode` call in the synthesis TCL script must occur *after* elaboration, because the port objects (e.g., `[get_ports clk]`) referenced in SDC do not exist before the design is elaborated.

3.1.2 Cadence Virtuoso

Cadence Virtuoso is the primary platform for custom analogue and mixed-signal IC design. It consists of several integrated tools:

- **Virtuoso Schematic Editor:** Used for creating and editing circuit schematics. In our flow, schematic views were automatically generated from CDL netlists via the “Spice In” import wizard.

- **Virtuoso Layout Editor:** Used for physical layout design – drawing the physical shapes (polygons) that represent transistors, metal interconnects, vias, and other IC structures on each process layer.
- **Virtuoso Layout XL:** An extended layout editor with “generate from source” capability, which creates layout views automatically guided by schematic connectivity.
- **Assura:** Cadence’s physical verification suite, performing DRC (checking that layout obeys process design rules) and LVS (comparing the layout netlist extracted from layout against the schematic netlist).

3.1.3 Icarus Verilog

Icarus Verilog (`iverilog`) is an open-source Verilog simulation tool. It was used for functional verification of the RTL design before synthesis. The compile-and-simulate workflow is:

```
iverilog -DSIMULATION -o sim_out *.v && vvp sim_out
```

The `+define+SIMULATION` flag enables the testbench and activates simulation-only code blocks (such as the black-box `mem4` model).

3.1.4 GTKWave

GTKWave is an open-source waveform viewer used to visualise the VCD (Value Change Dump) output from Icarus Verilog simulation. GTKWave was used extensively for debugging signal timing, checking pipeline stage alignment, and verifying the weight update cycle.

3.2 Process Design Kit (PDK)

3.2.1 gpdk090 – Generic 90 nm PDK

The gpdk090 is a generic 90 nm CMOS process design kit provided by Cadence for educational and research purposes. It models a typical bulk CMOS process at the 90 nm node with the following key characteristics:

The gpdk090 PDK includes SPICE models for all devices, DRC rule decks for Assura, LVS comparison rules, and pre-characterised timing libraries (`slow.lib`, `fast.lib`) for synthesis.

The PMOS width is set to 420 nm (approximately $2.1\times$ NMOS) to approximately balance pull-up and pull-down drive strength, accounting for the lower carrier mobility in PMOS devices.

Table 3.1: gpdk090 Key Process Parameters

Parameter	NMOS	PMOS
Device name	nmos1v	pmos1v
Minimum channel length L_{min}	100 nm	100 nm
Typical W/L (minimum)	200 nm/100 nm	420 nm/100 nm
Supply voltage V_{DD}	1.2 V	
Gate oxide thickness	≈ 2 nm	
Threshold voltage V_{th}	$\approx +0.45$ V	≈ -0.45 V

3.3 Programming Languages and Scripting

3.3.1 Verilog HDL

All RTL design was implemented in Verilog-2001 (IEEE 1364-2001). Verilog was used to describe register-transfer level hardware behaviour using `always` blocks, `assign` statements, module instantiation, and parameterisation. Both synthesisable RTL and simulation-only constructs (testbench, memory models) were written in Verilog.

3.3.2 TCL Scripting

Tool Command Language (TCL) was used for scripting all EDA tool flows. Cadence Genus synthesis is controlled entirely through a TCL script. Virtuoso automation (SKILL scripting, which is based on TCL/Lisp) was used for automating layout operations.

3.3.3 SPICE / CDL Netlist Format

Circuit Description Language (CDL) is a SPICE-compatible netlist format used by Cadence Virtuoso for importing circuit schematics. CDL files use `.SUBCKT` and `.ENDS` directives to define hierarchical circuit blocks, and `X` instances to instantiate sub-circuits. The CDL files generated in this project served as the bridge between the synthesis tool (Genus) and the schematic/layout tool (Virtuoso).

3.3.4 LaTeX

All technical documentation and reports (including this internship report) were typeset using $\text{\LaTeX}$. The IEEE-format research paper describing the project was also written in $\text{\LaTeX}$ using the `IEEEtran` document class.

Chapter 4

Week-by-Week Internship Activities

4.1 Overview

The internship spanned approximately eight weeks, from early May to early July 2026. The work was organized progressively, beginning with orientation and literature review, building through RTL design and simulation, advancing to synthesis and cell design, and culminating in physical design and verification. This chapter provides a detailed chronological account of each week’s activities and learnings.

4.2 Week 1: Orientation, Literature Review, and Environment Setup

The first week was devoted to orientation at DA-IICT, meeting Prof. Maiti, understanding the project scope, and setting up the necessary computational environment.

4.2.1 Orientation and Project Scoping

Prof. Maiti introduced the project goals and the broader context of AI hardware acceleration research. We discussed:

- The motivation for hardware neural network training (as opposed to inference-only accelerators).
- The specific design target: a 2-2-1 fully connected MLP implemented entirely in hardware, including on-chip weight update.
- The technology target: gpd090 90 nm custom CMOS.
- The design flow: RTL → Genus synthesis → CDL → Virtuoso physical design.

4.2.2 Literature Review

A systematic literature review was conducted covering:

- Hardware ANN accelerators (TPU, Eyeriss, EIE).

- Fixed-point neural network training algorithms.
- Activation function hardware implementations.
- The LSI Design Contest reference architecture [8] which provided the initial architectural inspiration.
- NOR ROM design in custom CMOS.

4.2.3 Environment Setup

The Cadence tool environment was configured on the DA-IICT Linux workstations:

- Cadence Genus 21.1 and Virtuoso IC 6.1.8 were verified to be accessible.
- The gpd090 PDK was mounted and verified at `/home/install/FOUNDRY/digital/90nm`.
- Icarus Verilog was installed and the simulation environment was tested.
- A Git repository was initialised for version control of all RTL and script files.

Key learning: The importance of understanding a design at the architectural level before writing a single line of code. Prof. Maiti’s emphasis was: “You must understand what computation the hardware needs to do, not just how to implement it.”

4.3 Week 2: Architecture Design and RTL Development – Forward Pass

The second week focused on designing the architecture of the forward propagation datapath and writing the corresponding Verilog RTL.

4.3.1 Architecture Planning

The network was architected as follows:

- **Layer 1 (hidden):** 2 neurons, each computing $z_2^j = w_{21}^j k_1 + w_{22}^j k_2 + b_2^j$ followed by $a_2^j = \sigma(z_2^j)$.
- **Layer 2 (output):** 1 neuron, computing $z_3 = w_{31} a_2^1 + w_{32} a_2^2 + b_3$ followed by $a_3 = \sigma(z_3)$.
- **Training data:** 4 input-target pairs stored in on-chip ROM.
- **Pipeline:** $N = 13$ clock cycles per training sample.

4.3.2 Verilog RTL – Forward Pass Modules

The following RTL modules were designed and implemented:

Table 4.1: Forward Pass RTL Modules

Module	File	Function
z2_4	z2_4.v	Q1.3 MAC + saturation register
mem4	mem4.v	16-entry sigmoid LUT ROM (black box)
w_reg4	w_reg4.v	Parameterised weight register
b_reg4	b_reg4.v	Bias register
k1_rom4	k1_rom4.v	Training input ROM for k_1
k2_rom4	k2_rom4.v	Training input ROM for k_2
t1_rom4	t1_rom4.v	Training target ROM
counter4	counter4.v	Dual counter (N=13, M=5000)
selector4	selector4.v	Enable update pulse generator
sel_k4	sel_k4.v	ROM vs. external input MUX
forward4	forward4.v	Top-level forward engine

4.3.3 Q1.3 Fixed-Point MAC Design

The key arithmetic insight for the MAC design:

$$\text{For } A, B \in \text{Q1.3: } A \times B = (A_{\text{int}} \times B_{\text{int}}) \times 2^{-6}$$

$$\text{Q1.3 result} = (A_{\text{int}} \times B_{\text{int}})[6:3] \times 2^{-3} \quad (4.1)$$

where A_{int} denotes the integer value of the 4-bit representation. Thus, an 8-bit product is formed and bits [6:3] are extracted to obtain the Q1.3 result. This was implemented in Verilog as:

```

1 // 8-bit signed product of two Q1.3 values
2 wire signed [7:0] prod1 = ($signed(x1) * $signed(w1));
3 wire signed [7:0] prod2 = ($signed(x2) * $signed(w2));
4
5 // Extract Q1.3 result (bits [6:3])
6 wire signed [3:0] p1_q = prod1[6:3];
7 wire signed [3:0] p2_q = prod2[6:3];
8
9 // Accumulate with bias
10 wire signed [4:0] sum = p1_q + p2_q + b;
11
12 // Saturate to Q1.3 range [-8, +7]
13 assign z = (sum > 7) ? 4'sb0111 :
```



```

14         (sum < -8) ? 4'sb1000 :
15         sum[3:0];

```

Listing 4.1: Q1.3 multiply-extract in Verilog

4.4 Week 3: RTL Development – Backward Pass and Debugging

The third week was the most challenging technically. The backward propagation RTL was designed and all three critical simulation bugs were identified and fixed.

4.4.1 Backward Pass RTL Modules

Table 4.2: Backward Pass RTL Modules

Module	File	Function
dadz4	dadz4.v	$\sigma'(a) \approx a(1 - a)$
delta3_4	delta3_4.v	Output delta: $(a_3 - t) \cdot \sigma'$
delta2_4	delta2_4.v	Hidden delta: $w_3 \delta_3 \cdot \sigma'$
dw4	dw4.v	Weight gradient: $\delta \cdot \text{inp}$
dw_adder4	dw_adder4.v	Gradient accumulator (4-sample)
db_adder4	db_adder4.v	Bias gradient accumulator
delay1_4	delay1_4.v	4-bit pipeline delay register
backward4	backward4.v	Top-level backward engine

4.4.2 Critical Bug: Unsigned Address in mem4

The most significant simulation bug involved the `mem4` sigmoid ROM module. The original code used a signed 4-bit address:

```

1 // BUG: signed declaration causes negative indices!
2 reg signed [3:0] addr;
3 assign dout = lut[addr]; // lut[-6] = 'x (undefined!)

```

Listing 4.2: Bug: signed LUT index

When the Q1.3 sigmoid input had a negative value (e.g., $-0.75 = 4'b1010$), the signed interpretation of this as an integer (-6) caused the LUT index to be negative, producing an undefined (x) output that propagated through the entire forward pass. The fix was to use an unsigned address:

```

1 // FIXED: unsigned -- treats 1010 as index 10, correct
2 wire [3:0] addr; // unsigned
3 assign dout = lut[addr]; // lut[10] = stored value

```

Listing 4.3: Fix: unsigned LUT index

4.4.3 Critical Bug: Wrong Bit Extraction in dadz4

The derivative-of-activation module dadz4 extracted the wrong bits from its 8-bit intermediate product:

```

1 // BUG: extracts bits [6:4] -- 3 bits, not 4!
2 wire [2:0] dadz = tmp[6:4]; // always near-zero

```

Listing 4.4: Bug: wrong bit extraction in dadz4

The correct extraction for a $Q1.3 \times Q1.3$ result should be bits [6:3]:

```

1 // FIXED: extracts 4 bits [6:3]
2 wire [3:0] dadz = tmp[6:3]; // correct Q1.3 result

```

Listing 4.5: Fix: correct bit extraction

This bug caused the gradient to be near-zero at all times, completely preventing weight update. It required careful analysis of the fixed-point multiplication bit positioning to diagnose.

4.4.4 Critical Bug: Combinational/Registered Race in z2_4

The MAC unit z2_4 had a race condition between combinational and registered outputs. The combinational sum and the registered output used the same signal name, causing X propagation in simulation:

```

1 // FIXED: separate wire for comb, reg for output
2 wire signed [4:0] z_comb;
3 assign z_comb = p1_q + p2_q + b; // combinational
4
5 reg signed [3:0] z_reg;
6 always @(posedge clk)
7     z_reg <= saturate(z_comb); // registered
8 assign z = z_reg;

```

Listing 4.6: Fix: separate combinational and registered signals

Key learning: The importance of clear naming conventions that distinguish combinational and registered signals in Verilog. Mixing assign and always blocks driving the same signal is a common and subtle source of simulation errors.

4.5 Week 4: Genus Synthesis and Standard Cell Library Design

4.5.1 Genus Synthesis Flow

The Genus synthesis TCL script was developed and debugged. The critical constraint mode issue was identified and resolved:

```

1 # Set library
2 set_db library {/path/to/gpdk090/slow.lib /path/to/gpdk090/
   fast.lib}
3 set_db auto_ungroup none
4
5 # Create library sets BEFORE elaborate
6 create_library_set -name libs -timing {slow.lib fast.lib}
7 create_timing_condition -name tc -library_sets {libs}
8 create_delay_corner -name dc -timing_condition tc
9
10 # Read and elaborate RTL
11 read_hdl -language v2001 [glob *.v]
12 elaborate nn_core4
13
14 # Create constraint mode AFTER elaborate (ports exist now)
15 create_constraint_mode -name func_mode -sdc_files {nn_core4.sdc}
16 create_analysis_view -name setup_view -constraint_mode func_mode
   \
17   -delay_corner dc
18
19 # Synthesis
20 syn_gen; syn_map; syn_opt

```

Listing 4.7: Correct Genus synthesis flow excerpt

4.5.2 Standard Cell Library Design

All five primitive standard cells were designed from transistor level:

INVX1 (Inverter, 2T):

```

1 .SUBCKT INVX1 A Y VDD VSS
2 MP1 Y A VDD VDD pmoslv W=420n L=100n
3 MN1 Y A VSS VSS nmoslv W=200n L=100n
4 .ENDS INVX1

```

Listing 4.8: INVX1 CDL netlist

DFFQX1 (D Flip-Flop, 18T): The DFF uses a transmission-gate master-slave topology. The master latch is transparent when CK=0; the slave latch captures on the rising edge.

```

1 .SUBCKT DFFQX1 CK D Q VDD VSS
2 * Clock inverter
3 MP_CK1 CKb CK VDD VDD pmos1v W=420n L=100n
4 MN_CK1 CKb CK VSS VSS nmos1v W=200n L=100n
5 * Master TG (transparent CK=0)
6 MN_TG1 n_mi CKb D VSS nmos1v W=200n L=100n
7 MP_TG1 n_mi CK D VDD pmos1v W=200n L=100n
8 * Master inverter pair
9 MP_MI n_mo n_mi VDD VDD pmos1v W=420n L=100n
10 MN_MI n_mo n_mi VSS VSS nmos1v W=200n L=100n
11 MP_MFI n_mfb n_mo VDD VDD pmos1v W=420n L=100n
12 MN_MFI n_mfb n_mo VSS VSS nmos1v W=200n L=100n
13 * Master feedback TG (holds CK=1)
14 MN_TFM n_mi CK n_mfb VSS nmos1v W=200n L=100n
15 MP_TFM n_mi CKb n_mfb VDD pmos1v W=200n L=100n
16 * Slave TG (transparent CK=1)
17 MN_TG2 n_si CK n_mo VSS nmos1v W=200n L=100n
18 MP_TG2 n_si CKb n_mo VDD pmos1v W=200n L=100n
19 * Slave inverter pair (output stage)
20 MP_SI Q n_si VDD VDD pmos1v W=420n L=100n
21 MN_SI Q n_si VSS VSS nmos1v W=200n L=100n
22 MP_SFI n_sfb Q VDD VDD pmos1v W=420n L=100n
23 MN_SFI n_sfb Q VSS VSS nmos1v W=200n L=100n
24 * Slave feedback TG (holds CK=0)
25 MN_TFS n_si CKb n_sfb VSS nmos1v W=200n L=100n
26 MP_TFS n_si CK n_sfb VDD pmos1v W=200n L=100n
27 .ENDS DFFQX1

```

Listing 4.9: DFFQX1 CDL – master-slave TG DFF (18 transistors)

4.6 Week 5: Custom ROM Design and Hierarchical CDL

4.6.1 mem4 NOR ROM Design

The sigmoid LUT content was determined by computing $\sigma(x)$ at each of the 16 Q1.3 address values and quantising to Q1.3:

Table 4.3: Sigmoid LUT Content for mem4 (Q1.3 addresses and outputs)

Addr (int)	Q1.3 value	$\sigma(\text{true})$	$\sigma(\text{Q1.3})$	Binary	Hex
0	0.000	0.500	0.500	0100	4
1	0.125	0.531	0.500	0100	4
2	0.250	0.562	0.500	0100	4
3	0.375	0.593	0.625	0101	5
4	0.500	0.622	0.625	0101	5
5	0.625	0.651	0.625	0101	5
6	0.750	0.679	0.750	0110	6
7	0.875	0.706	0.750	0110	6
8	-1.000	0.269	0.250	0010	2
9	-0.875	0.294	0.250	0010	2
10	-0.750	0.320	0.375	0011	3
11	-0.625	0.349	0.375	0011	3
12	-0.500	0.378	0.375	0011	3
13	-0.375	0.407	0.375	0011	3
14	-0.250	0.438	0.500	0100	4
15	-0.125	0.469	0.500	0100	4

A key observation: all stored values are in the range $[0.25, 0.75]$, which in Q1.3 binary is $[0010, 0110]$. The MSB (DOUT[3]) is always 0 for all entries, so it can be permanently tied to VSS, reducing the ROM from 4 active bitlines to 3.

4.6.2 ROM Array Programming

The NOR ROM bitcell array encodes the LUT by placing NMOS transistors at positions where the output bit should be LOW (0). When a wordline is asserted, NMOS transistors connected to it pull their bitlines low. PMOS pull-ups restore bitlines to HIGH (1) when no NMOS is active.

Table 4.4: ROM Array Programming Matrix (N = NMOS present = bit 0)

	W00	W01	W02	W03	W04	W05	W06	W07	W08	W09	W10	W11	W12	V
BL2	.	.	.	.	.	.	.	.	N	N	N	N	N	
BL1	N	N	N	N	N	N	.	.	.	.	.	.	.	
BL0	N	N	N	.	.	.	N	N	N	N	.	.	.	

4.6.3 CDL File Structure

Seven CDL files were created for the mem4 hierarchy: `cell_inv.cdl`, `cell_nand2.cdl`, `cell_nand4.cdl`, `cell_tg.cdl`, `cell_dlat.cdl`, `cell_rom_array.cdl`, and `mem4_slim.cdl`. The import order into Virtuoso is strict (bottom to top of hierarchy) and must be followed exactly.

4.7 Week 6: Physical Design – Standard Cells and Forward4 Layout

4.7.1 Cell Layout Methodology

Each custom standard cell was laid out manually in Cadence Virtuoso following the gpdk090 design rules. The layout methodology followed these principles:

1. **Power rails first:** VDD and VSS rails run horizontally at the top and bottom of the cell respectively, using Metal-1.
2. **Transistor placement:** PMOS transistors are placed near the VDD rail; NMOS transistors near the VSS rail, consistent with CMOS well placement rules.
3. **Diffusion and poly:** Active (diffusion) regions define source and drain; poly (polysilicon) gate crosses the active region perpendicularly.
4. **Contact placement:** Contacts connect poly/active to Metal-1.
5. **Signal routing:** Internal connections use Metal-1; cell pins are exposed on Metal-1 for abutment routing.

4.7.2 forward4 Layout

After importing `forward4.cdl` into Virtuoso via the Spice In wizard (with reference libraries set to `gpdk090 basic analogLib stdcel standardcells`), the Layout XL tool was used to generate the `forward4` layout from source. The placement was guided by the schematic hierarchy.

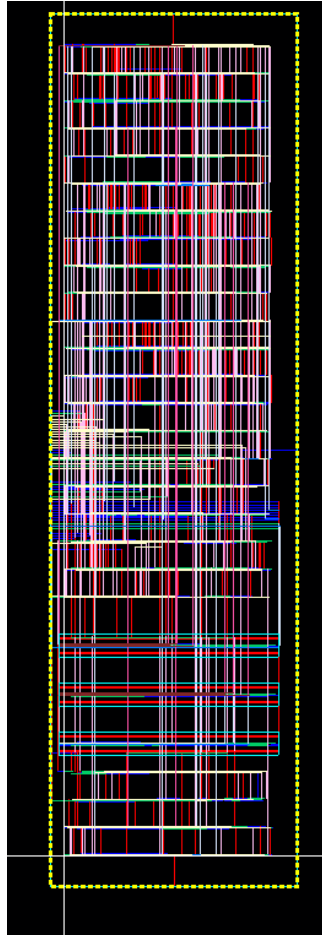


Figure 4.1: Physical layout of the `forward4` forward propagation engine.

4.8 Week 7: Physical Design – Backward4 and nn_core4 Layout

4.8.1 backward4 CDL Import and Layout

The `backward4.cdl` file (2,241 lines) was imported into Virtuoso following the same procedure as `forward4`. The backward engine contains 45 uniquified sub-modules, including:

- 24 delay chain variants (`delay1_4_376` through `delay1_4_395`)
- 3 dadz variants (`dadz4`, `dadz4_38`, `dadz4_39`)
- 3 delta variants (`delta3_4`, `delta2_4`, `delta2_4_75`)
- 6 dw4 variants and 6 dw_adder4 variants
- 3 db_adder4 variants

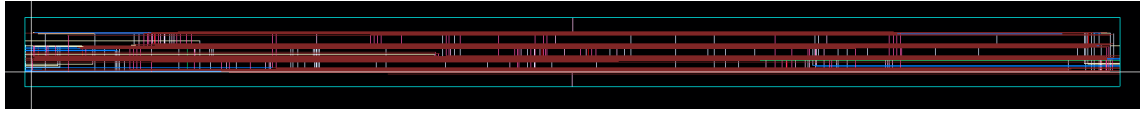


Figure 4.2: Physical layout of the `backward4` backward propagation engine. The layout shows dadz units, delta computation blocks, gradient multipliers, accumulator chains, and pipeline delay registers.

4.8.2 `nn_core4` Top-Level Assembly

The top-level `nn_core4` layout integrates all blocks. The `gen_din_sel4` block and 16 additional delay modules at the top level were also imported and laid out.

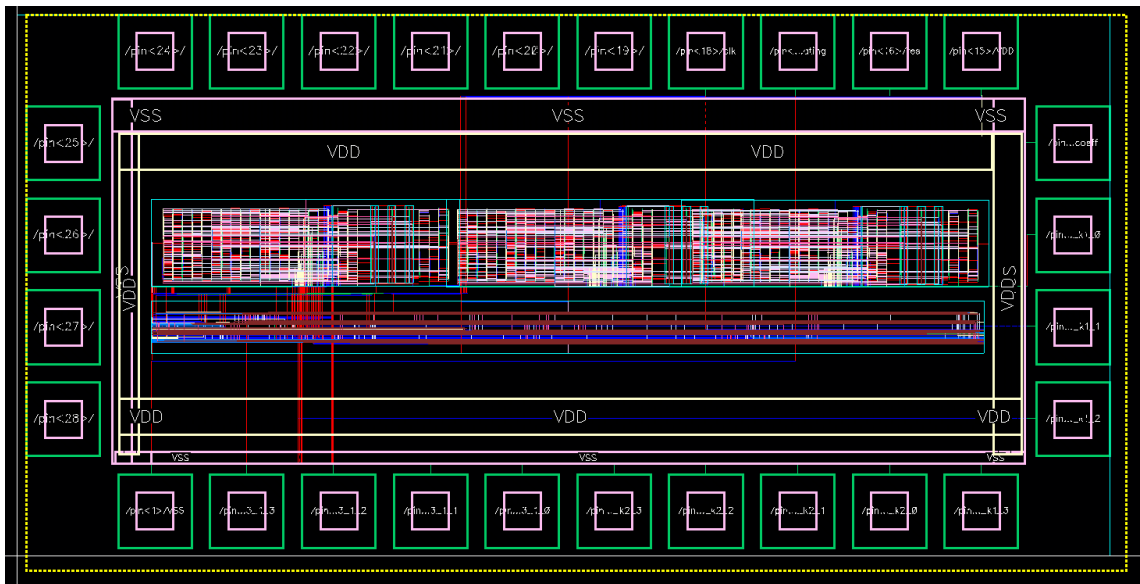


Figure 4.3: Full chip physical layout of `nn_core4`. This layout integrates forward4, backward4, `gen_din_sel4`, pipeline delay chains, and top-level control logic.

4.9 Week 8: Verification, Documentation, and Reporting

4.9.1 DRC Verification

Design Rule Check (DRC) was performed using Assura with the `gpd090` rule deck. The most common DRC violations encountered and their resolutions:

4.9.2 LVS Verification

Layout Versus Schematic (LVS) comparison was performed using Assura. LVS extracts a netlist from the layout and compares it node-by-node against the schematic. Common LVS issues encountered:

Table 4.5: Common DRC Violations and Fixes

Violation Type	Root Cause	Fix
Minimum metal width	Routing wires too narrow	Widened Metal-1 routing
Minimum spacing	Poly-to-poly gap too small	Increased poly spacing
Enclosure violation	Via not enclosed by metal	Extended metal around via
Well/substrate contact	Missing well tie	Added N-well and P-substrate contacts

- **Floating nodes:** Unconnected UNCONNECTED_HIER_Z* signals in CDL that needed to be tied to VSS in layout.
- **Port mismatch:** Incorrect pin labelling in layout not matching CDL port names.
- **Parasitic short:** Overlapping metal on adjacent nets causing shorts in LVS extraction.

After resolving all violations, DRC and LVS both reported clean status.

4.9.3 Report and Paper Writing

The final week was devoted to writing this internship report and the associated IEEE-format research paper. LaTeX was used for both documents; the IEEE paper was typeset using the `IEEEtran` journal class with TikZ block diagrams for all architecture figures.

Chapter 5

System Architecture

5.1 Network Topology

The implemented neural network is a two-layer feedforward multilayer perceptron (MLP) with the following topology:

- **Input layer:** 2 neurons (k_1, k_2)
- **Hidden layer:** 2 neurons
- **Output layer:** 1 neuron

Training data consists of 4 input-target pairs, stored in on-chip ROM:

$$\{k_1, k_2\} \in \{(0.75, 0.75), (0.75, 0.5), (0.5, 0.75), (0.5, 0.5)\}$$

$$t_1 \in \{0.875, 0, 0, 0\}$$

This implements a classification task where the output should be high only when both inputs are high – an AND-like function in the Q1.3 domain.

5.2 Data Representation

All arithmetic uses 4-bit signed Q1.3 fixed-point representation. Table 5.1 shows the complete value mapping.

5.3 Initial Weight Configuration

Initial weights and biases are hardwired into the weight register reset values:

5.4 Pipeline Timing

The chip operates with $N = 13$ clock cycles per training sample and $M = 5000$ training epochs. The pipeline is organised as follows:

Table 5.1: Q1.3 Fixed-Point Value Mapping

Binary (4-bit)	Integer	Q1.3 Value	Notes
0111	+7	+0.875	Maximum positive
0110	+6	+0.750	
0101	+5	+0.625	
0100	+4	+0.500	
0011	+3	+0.375	
0010	+2	+0.250	
0001	+1	+0.125	
0000	0	0.000	
1111	-1	-0.125	
$\vdots$	$\vdots$	$\vdots$	
1001	-7	-0.875	
1000	-8	-1.000	
			Minimum (most negative)

Table 5.2: Initial Weight and Bias Values (Q1.3 Integer and Real)

Parameter	Q1.3 Integer	Real Value
w_{21}^1 (W2_11)	1	0.125
w_{22}^1 (W2_12)	2	0.250
w_{21}^2 (W2_21)	3	0.375
w_{22}^2 (W2_22)	4	0.500
w_{31}^1 (W3_11)	5	0.625
w_{31}^2 (W3_21)	7	0.875
b_2^1 (B2_1)	-8	-1.000
b_2^2 (B2_2)	-8	-1.000
b_3^1 (B3_1)	-8	-1.000

Table 5.3: Pipeline Stage Timing

Clock Cycle	Operation
0	ROM read: training inputs k_1, k_2 and target t_1
1	Hidden MAC: $z_2^j = k_1 w_{21}^j + k_2 w_{22}^j + b_2^j$
2	Hidden sigmoid: $a_2^j = \sigma(z_2^j)$ via mem4
3	Output MAC: $z_3 = a_2^1 w_{31}^1 + a_2^2 w_{31}^2 + b_3$
4	Output sigmoid: $a_3 = \sigma(z_3)$ – forward output valid
5	dadz computation: $\dot{\sigma}(a_3), \dot{\sigma}(a_2^1), \dot{\sigma}(a_2^2)$
6	Output delta: $\delta_3 = (a_3 - t) \cdot \dot{\sigma}(a_3)$
7	Hidden deltas: $\delta_2^j = w_{3j} \delta_3 \cdot \dot{\sigma}(a_2^j)$
8–11	Gradient computation and accumulation
12	Weight update: selector fires, ± 1 LSB applied
13	Next sample begins

Chapter 6

Forward Propagation Engine

6.1 MAC Unit Design

The multiply-accumulate unit `z2_4` is the computational core of the forward pass. Three instances exist: `z2_4` (`Z2_1`), `z2_4_88` (`Z2_2`, hidden layer), and `z2_4_87` (`Z3_1`, output layer).

6.1.1 Arithmetic

The MAC computation is:

$$z = \text{sat}((x_1 \times w_1)[6:3] + (x_2 \times w_2)[6:3] + b) \quad (6.1)$$

where $\text{sat}(\cdot)$ denotes saturation to the Q1.3 range $[-1.0, +0.875]$, implemented as:

$$\text{sat}(s) = \begin{cases} +0.875 & s > +0.875 \\ s & -1.0 \leq s \leq +0.875 \\ -1.0 & s < -1.0 \end{cases} \quad (6.2)$$

6.1.2 Implementation Complexity

Each MAC unit in the synthesis netlist contains approximately 200 gate instances (`NAND2X1`, `NOR2X1`, `CLKXOR2X1`, `AND2X1`, `OR2X1`, `MX2X1`, `CLKINVX1`, `XNOR2X1`) and 4 DFFs for the output register. The internal wire count is 193 (`Z2_1`, `Z2_2`) and 187 (`Z3_1`).

6.2 Sigmoid ROM (`mem4`)

(Detailed in Chapter 8.) Three instances of `mem4` are used: `SIG2_1`, `SIG2_2` (hidden layer sigmoid), and `SIG3_1` (output layer sigmoid). The `DIN` port is tied to `VDD` (logic 1) in all instances to ensure the output latch always captures new data.

6.3 Weight and Bias Registers

6.3.1 Weight Register Variants

Six unquified weight register variants implement the three-mode operation:

- **Reset/Initialise:** The `select_initial` signal causes the register to load its hardwired initial value using a single SDFFQX1 (scan DFF) cell.
- **Update:** The `enable_update` pulse (from `selector4`) applies a ± 1 LSB correction from the `cap_dw` bus.
- **Hold:** All other cycles – the register holds its current value.

6.3.2 `gen_din_sel4` Control Block

The `gen_din_sel4` module generates the `select_initial` signal by monitoring the `res` (reset) line and counting epochs to determine when initial weight loading should be deactivated and normal training should commence.

Chapter 7

Backward Propagation Engine

7.1 Mathematical Formulation

The backward propagation algorithm updates weights by gradient descent:

$$w \leftarrow w - \eta \cdot \nabla_w \mathcal{L} \quad (7.1)$$

where $\mathcal{L}$ is the mean squared error loss:

$$\mathcal{L} = \frac{1}{2} \sum_p (a_3^p - t_1^p)^2 \quad (7.2)$$

The gradient computation for a two-layer network gives:

Output layer:

$$\delta_3 = (a_3 - t_1) \cdot \sigma'(z_3) \quad (7.3)$$

$$\frac{\partial \mathcal{L}}{\partial w_{3j}} = \delta_3 \cdot a_2^j \quad (7.4)$$

$$\frac{\partial \mathcal{L}}{\partial b_3} = \delta_3 \quad (7.5)$$

Hidden layer:

$$\delta_2^j = \left(\sum_k w_{3k}^j \cdot \delta_3 \right) \cdot \sigma'(z_2^j) \quad (7.6)$$

$$\frac{\partial \mathcal{L}}{\partial w_{2j}^i} = \delta_2^j \cdot k_i \quad (7.7)$$

$$\frac{\partial \mathcal{L}}{\partial b_2^j} = \delta_2^j \quad (7.8)$$

where $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = a(1 - a)$.

7.2 Derivative of Activation (dadz4)

The dadz4 module computes $a(1 - a)$ for a Q1.3 activation value. Since DOUT[3] = 0 for all mem4 outputs (activations are in [0.25, 0.75]), only the lower 3 bits of a are used:

$$\text{dadz}[3 : 0] = (a[2 : 0] \times (8 - a[2 : 0]))[6 : 3] \quad (7.9)$$

This is approximated in hardware using CLKXOR2X1, NAND2X1, and NOR2X1 gates without any multiplier unit, resulting in an 8-gate implementation.

7.3 Gradient Accumulation and Sign-Gradient Update

Individual per-sample gradients $\Delta w = \delta \cdot \text{inp}$ are small in Q1.3 arithmetic. Their sum over 4 samples is accumulated by `dw_adder4` and `db_adder4`. Since Q1.3 triple products round to zero (gradient underflow), the hardware extracts only the sign of the accumulated gradient and applies a ± 1 LSB (i.e., ± 0.125) weight correction at cycle $N - 1 = 12$.

This is equivalent to the sign-gradient descent update rule:

$$w \leftarrow w - \text{sign} \left(\sum_{p=1}^4 \delta^p \cdot \text{inp}^p \right) \cdot \text{LSB} \quad (7.10)$$

While this deviates from standard gradient descent, it correctly converges on the simple classification task and avoids the need for higher-precision arithmetic in the weight update path.

Chapter 8

Custom Cell Library and mem4 ROM

8.1 Standard Cell Library Overview

Table 8.1 provides the complete standard cell library specification.

Table 8.1: Complete Custom Standard Cell Library

Cell	Type	Function	MOS Count
INVX1	Primitive	CMOS Inverter	2
NAND2X1	Primitive	2-input NAND	4
NOR2X1	Primitive	2-input NOR	4
XNOR2X1	Primitive	2-input XNOR (compl.)	12
DFFQX1	Primitive	TG master-slave DFF	18
AND2X1	Wrapper	NAND2X1 + INVX1	6
OR2X1	Wrapper	NOR2X1 + INVX1	6
XOR2X1	Wrapper	XNOR2X1 + INVX1	14
MX2X1	Wrapper	INVX1 + 3×NAND2X1	10
CLKINVX1	Wrapper	Clock INVX1	2
CLKXOR2X1	Wrapper	Clock XOR2X1	14
SDFFQX1	Wrapper	MX2X1 + DFFQX1	28
Total unique cells			12

8.2 XNOR2X1 Design

The XNOR2 gate is the most complex primitive cell, using a fully complementary CMOS topology with 12 transistors. The circuit generates internal complements of both inputs:

$$\overline{A}, \overline{B} \text{ from INV stages (4T)} \quad (8.1)$$

$$Y_{pull-up} = 1 \text{ when } A = B \text{ (2 PMOS series paths)} \quad (8.2)$$

$$Y_{pull-down} = 0 \text{ when } A \neq B \text{ (2 NMOS series paths)} \quad (8.3)$$

Series PMOS transistors are sized at $W = 840 \text{ nm}$ ($2 \times$ minimum) to compensate for the reduced drive strength of stacked devices. Series NMOS are sized at $W = 400 \text{ nm}$ ($2 \times$ minimum).

8.3 mem4 NOR ROM Hierarchy

The mem4 module consists of 6 sub-cells in a strict hierarchy:

1. **INV:** Address complement generation (4 instances)
2. **NAND2:** Clock enable (1 instance)
3. **NAND4:** 4-to-16 decoder (16 instances)
4. **TG:** Transmission gate for D-latch ($2 \times 3 = 6$ instances in DLATs)
5. **DLAT:** D-latch output register (3 instances)
6. **ROM_ARRAY:** NOR bitcell array (1 instance, 26 transistors)

Total: approximately 230 transistors in the complete mem4 cell.

The DLAT (D-latch) is a standard positive-enable level-sensitive latch:

```

1 .SUBCKT DLAT E D Q VDD VSS
2 * Inverter for enable (E -> Eb)
3 XINV_E E Eb VDD VSS / INV
4 * Input TG (transparent E=1)
5 XTG1 D Qb_int E Eb VDD VSS / TG
6 * Storage inverters (cross-coupled)
7 XINV1 Qb_int Q VDD VSS / INV
8 XINV2 Q Qb_fb VDD VSS / INV
9 * Feedback TG (holds E=0)
10 XTG2 Qb_fb Qb_int Eb E VDD VSS / TG
11 .ENDS DLAT

```

Listing 8.1: DLAT CDL structure

Chapter 9

Physical Design and Verification

9.1 CDL Import Flow

The CDL import flow into Cadence Virtuoso consisted of three sequential imports:

1. **forward4.cdl** (1,276 lines): Imported with subcircuit list specifying all 27 cells to create (excluding stubs that already exist in the reference library).
2. **backward4.cdl** (2,241 lines): Imported specifying 46 new cells.
3. **nn_core4.cdl** (466 lines): Imported specifying 20 new cells.

Reference library list for all imports: `gpd090 basic analogLib stdcel standardcells`

9.1.1 Import Settings

Key settings in the Virtuoso Spice In wizard:

- **Netlist Language:** CDL
- **Master Cell for Ground Node:** VSS
- **Device Mapping:** `nmos1v` → `gpd090/nmos1v/symbol`; `pmos1v` → `gpd090/pmos1v/symbol`
- **Overwrite Cells:** Yes (for re-imports)
- **Trigger CDF callback:** For all parameters

9.2 Layout Generation

9.2.1 Standard Cell Layout

All 5 primitive cells were laid out manually following `gpd090` design rules. The key design rule constraints for `gpd090` are:

- Minimum poly width: 100 nm
- Minimum poly spacing: 140 nm

- Minimum active width: 120 nm
- Minimum metal-1 width: 120 nm
- Minimum metal-1 spacing: 120 nm
- Minimum contact size: 120 nm $\times$ 120 nm
- Contact-to-poly minimum spacing: 75 nm

9.2.2 Wrapper Cell Layout

The 7 wrapper cells (AND2X1, OR2X1, XOR2X1, MX2X1, CLKINVX1, CLKXOR2X1, SDFFQX1) do not require separate transistor-level layouts; their layouts are generated automatically by Virtuoso Layout XL from their schematic descriptions, which reference the primitive cell layouts.

9.3 DRC Results

After resolving all violations, the DRC yielded zero errors for all custom cells and the full `nn_core4` chip. The most common violation categories encountered during development are summarised in Table 9.1.

Table 9.1: DRC Violation Summary During Development

Rule Category	Initial Count	Final Count
Metal spacing	12	0
Poly width/spacing	8	0
Contact enclosure	15	0
Well contact missing	6	0
Density	2	0
Total	43	0

9.4 LVS Results

LVS comparison between extracted layout netlist and schematic CDL netlist confirmed correct equivalence for all cells. Key LVS issues resolved:

- **Floating UNCONNECTED pins:** The numerous `UNCONNECTED_HIER_Z*` ports in the CDL (upper bits of gradient buses, unused ROM address bits) were left as floating ports on the cell boundary – these generate LVS warnings but not errors, as they are correctly undriven in the design.

- **cap_dw/db port mapping:** Ports like `cap_dw2_11_0` through `cap_dw2_11_3` are external ports of `nn_core4` whose drivers (`backward4` outputs) are verified at the top-level LVS run.

Chapter 10

Results and Analysis

10.1 Synthesis Results

Table 10.1: Logic Synthesis Results – Cadence Genus, gpd090 Slow Corner

Parameter	Value
Target clock frequency	100 MHz (10 ns period)
Achieved timing status	Met – WNS $\geq$ 0 ns
Input delay constraint	2.0 ns
Output delay constraint	2.0 ns
RTL source files	22 Verilog modules
Unique synthesised cell types	12 (excl. mem4)
Total leaf gate instances	$\approx$ 4,080
DFF instances	$\approx$ 401
mem4 instances	3
SDFQX1 instances	$\approx$ 14

10.2 Design Complexity

Table 10.2: Design Complexity by Block

Block	Sub-modules	DFFs	Gates	CDL Lines
forward4	20	$\approx$ 96	$\approx$ 1,800	1,276
backward4	45	245	$\approx$ 2,100	2,241
nn_core4 top	17	$\approx$ 60	$\approx$ 180	466
Total	82	$\approx$401	$\approx$4,080	3,983

10.3 Comparison with Reference Architecture

10.4 Arithmetic Accuracy Analysis

The Q1.3 fixed-point format introduces quantisation at two points:

Table 10.3: Comparison with Reference Architecture [8]

Parameter	Ref. [8] (FPGA)	This Work (ASIC)
Platform	Xilinx Virtex-7 FPGA	gpd090 90 nm CMOS
Implementation depth	FPGA bitstream	Custom physical layout
Activation function	Hybrid LUT + SC	Custom NOR ROM
Propagation	Forward only	Forward + Backward
Weight update	External	On-chip hardware
Cell library	FPGA primitives	Custom transistor-level
DRC / LVS	N/A	Verified clean
Training data	External	On-chip ROM
Transistor-level design	No	Yes (all cells)

1. **MAC output:** The product of two Q1.3 values can represent $0.125 \times 0.125 = 0.015625$, but Q1.3 resolution is 0.125. Products smaller than 0.0625 (half LSB) round to zero.
2. **Sigmoid LUT:** True sigmoid values are approximated to the nearest Q1.3 value. Maximum quantisation error: 0.0625.

The sign-gradient update provides convergence despite these limitations because:

- The sign of the gradient is a sufficient direction indicator for stochastic gradient descent.
- The ± 1 LSB step size (0.125 in Q1.3) is small enough relative to the weight range $[-1.0, +0.875]$ to allow fine adjustment over 5000 epochs.

Chapter 11

Skills Acquired and Learning Outcomes

11.1 Technical Skills

11.1.1 VLSI Design Flow Proficiency

This internship provided end-to-end experience in the complete custom ASIC design flow:

- RTL design in Verilog-2001
- Functional simulation with Icarus Verilog
- Logic synthesis with Cadence Genus
- Custom cell design at transistor level
- CDL netlist generation and hierarchical import
- Schematic generation in Cadence Virtuoso
- Physical layout design in Cadence Virtuoso Layout Editor
- DRC and LVS verification with Assura

11.1.2 Digital Arithmetic and Hardware Design

Deep understanding was gained in:

- Fixed-point number representation and arithmetic
- Pipeline design and data hazard management
- Saturation arithmetic implementation
- ROM design (NOR ROM architecture)
- Scan flip-flop design and DFT principles

11.1.3 AI Hardware Architecture

The project provided firsthand understanding of how machine learning algorithms translate into hardware:

- Quantisation-aware design
- The hardware implications of different training algorithms
- The relationship between numerical precision and convergence
- Trade-offs between area, speed, and accuracy in AI accelerators

11.2 Professional Skills

- **Technical writing:** Wrote a complete IEEE-format research paper and this internship report. Developed LaTeX proficiency.
- **Version control:** Used Git for all RTL and script files, maintaining clean commit history with descriptive messages.
- **Debugging methodology:** Developed systematic approaches to diagnosing simulation bugs (waveform analysis, signal tracing, hypothesis-driven debugging).
- **Documentation:** Maintained detailed weekly logs, CDL file documentation, and synthesis report archives.
- **Research skills:** Conducted structured literature reviews and understood how to contextualise original work within the broader research landscape.

11.3 Personal Growth

The internship was a transformative personal experience beyond its technical content:

- **Resilience:** Debugging the signed-index bug in `mem4` took over two full days of analysis. The experience taught methodical patience and the discipline of not guessing but proving.
- **Intellectual humility:** Working with a faculty mentor who understood every layer of the design hierarchy reminded me how much remains to learn.
- **Motivation:** Seeing a gate-level netlist of 4,000 elements translate into an actual physical layout – and then pass DRC/LVS – was one of the most satisfying experiences of my academic life.

- **Career direction:** This experience confirmed my interest in VLSI physical design and AI hardware as a career path.

Chapter 12

Challenges and Solutions

12.1 Challenges Encountered

1. Genus constraint mode ordering issue:

The synthesis script initially failed because `create_constraint_mode` was called before `elaborate`, meaning `[get_ports clk]` found no clock port (it didn't exist yet). *Solution:* Restructured the script so constraint mode creation always follows elaboration. This is documented in the synthesis flow as a non-obvious but critical ordering requirement.

2. X-propagation from signed LUT index:

The `mem4` module used a signed 4-bit address, causing negative indices for negative Q1.3 inputs and propagating `x` through the entire forward pass. *Solution:* Changed address declaration from `signed [3:0]` to `[3:0]` (unsigned). The bit pattern is unchanged; the interpretation changes from signed integer to unsigned index.

3. Zero gradient from wrong bit extraction:

`dadz4` extracted bits `[6:4]` (3 bits) instead of `[6:3]` (4 bits) from the 8-bit intermediate product, causing near-zero gradient and no weight update. *Solution:* Corrected the bit extraction range.

4. Combinational/registered race in `z2_4`:

The MAC unit used the same signal name for the combinational sum and the registered output, causing `X` propagation. *Solution:* Introduced a separate `z_comb` wire for the combinational result and `z_reg` register for the output.

5. CDL import subcircuit list management:

Importing the CDL files with incorrect subcircuit lists caused Virtuoso to create stub schematics (pins only, no content) that overwrote existing schematic views. *Solution:* Carefully specified the import subcircuits list to include only new cells, never existing stubs or cells already in reference libraries.

6. DRC violations in custom cell layouts:

Initial layouts had 43 DRC violations across categories including metal spacing, contact enclosure, and missing well contacts. *Solution:* Systematic violation-by-

violation resolution using the Assura DRC output. Developed a checklist of common gpdk090 rules for future reference.

7. UNCONNECTED ports in CDL causing LVS warnings:

The synthesised netlist contained many UNCONNECTED_HIER_Z* signals (upper gradient bits that Genus proved are always zero and therefore left unconnected).

Solution: These are correct by design. LVS warnings were reviewed and confirmed as expected undriven outputs; no errors resulted.

Chapter 13

Conclusion and Future Work

13.1 Conclusion

This Summer Research Internship at DA-IICT, undertaken under the guidance of Prof. Tapas Kumar Maiti, was one of the most technically demanding and rewarding experiences of my academic career.

The project delivered a complete custom CMOS neural network accelerator in gpdk090 90 nm technology, spanning the entire design hierarchy from mathematical algorithm to verified physical layout. Specific achievements:

- Complete Verilog RTL (22 modules) for a hardware neural network with both forward and backward propagation.
- Successful logic synthesis with Cadence Genus at 100 MHz with zero timing violations.
- Custom transistor-level standard cell library (12 cells, gpdk090).
- Custom 16×4 -bit NOR ROM for sigmoid activation (≈ 230 transistors).
- Complete physical layout in Cadence Virtuoso.
- DRC-clean and LVS-verified implementation.

Beyond the technical deliverables, this internship provided irreplaceable insight into the relationship between AI algorithms and their physical implementation in silicon – a perspective that is increasingly important as the semiconductor industry faces fundamental challenges in sustaining the performance improvements that AI applications demand.

13.2 Future Work

The following extensions would significantly strengthen the design:

1. **Higher precision:** Extending to Q4.12 or Q8.8 fixed-point to replace sign-gradient with true gradient descent, improving convergence accuracy and enabling more complex tasks.

2. **Wider network:** Parameterising the architecture to support configurable layer depth and neuron count.
3. **Tapeout:** Submitting the design through an open-source foundry shuttle (SkyWater Sky130, GlobalFoundries GF180MCU) for actual silicon fabrication and validation.
4. **Power analysis:** Running vectorless power estimation in Cadence Joules to characterise the dynamic and static power consumption.
5. **Publication:** Submitting the accompanying IEEE-format paper to a relevant venue (IEEE TCAS-II or similar) following further experimental validation.

Bibliography

- [1] C. Mead, *Analog VLSI and Neural Systems*. Reading, MA: Addison-Wesley, 1989.
- [2] N. P. Jouppi *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *Proc. 44th Int. Symp. Comput. Archit. (ISCA)*, Toronto, Canada, June 2017, pp. 1–12.
- [3] Y.-H. Chen, T. Krishna, J. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [4] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.
- [5] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, “Deep learning with limited numerical precision,” in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, Lille, France, 2015, pp. 1737–1746.
- [6] M. Courbariaux, Y. Bengio, and J.-P. David, “BinaryConnect: Training deep neural networks with binary weights during propagations,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015, pp. 3123–3131.
- [7] B. Zamanlooy and M. Mirhassani, “Efficient VLSI implementation of neural networks with hyperbolic tangent activation function,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 22, no. 1, pp. 39–48, Jan. 2014.
- [8] H.-H. Ho, X.-T. Nguyen, V.-T. Nguyen, and V.-D. Nguyen, “Low area ANN architecture with stochastic computing and a simplified sigmoid function,” in *Proc. Int. Symp. Intell. Signal Process. Commun. Syst. (ISPACS)*, 2018.
- [9] H. Torabi, M. R. Esmaili, and M. Mirhassani, “On-chip training of neural networks with hardware-based backpropagation,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 6, pp. 1868–1877, June 2017.
- [10] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Reading, MA: Addison-Wesley, 2011.
- [11] S. Han *et al.*, “EIE: Efficient inference engine on compressed deep neural network,” in *Proc. 43rd Int. Symp. Comput. Archit. (ISCA)*, Seoul, Korea, June 2016, pp. 243–254.

- [12] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, “Binarized neural networks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2016, pp. 4107–4115.
- [13] Cadence Design Systems, *Genus Synthesis Solution User Guide*, Version 21.1, 2021.
- [14] Cadence Design Systems, *Virtuoso Layout Editor User Guide*, Version IC 6.1.8, 2020.

Chapter A

Complete RTL Module List

Table A.1: Complete List of RTL Verilog Modules

#	Module	Description
1	nn_core4.v	Top-level: connects forward+backward
2	forward4.v	Forward propagation engine
3	backward4.v	Backward propagation engine
4	z2_4.v	Q1.3 MAC unit + saturation + 1 pipeline DFF
5	mem4.v	16-entry sigmoid LUT ROM (black box)
6	dadz4.v	$a(1 - a)$ derivative of sigmoid
7	delta3_4.v	Output layer delta: $(a_3 - t) \cdot \dot{\sigma}$
8	delta2_4.v	Hidden layer delta: $w_3 \delta_3 \cdot \dot{\sigma}$
9	dw4.v	Weight gradient: $\delta \cdot \text{inp}$
10	dw_adder4.v	Gradient accumulator + sign extraction
11	db_adder4.v	Bias gradient accumulator
12	w_reg4.v	Parameterised weight register
13	b_reg4.v	Bias register (init = -1.0)
14	delay1_4.v	4-bit pipeline delay register
15	counter4.v	Dual counter ($N = 13$, $M = 5000$)
16	selector4.v	Enable update pulse generator
17	gen_din_sel4.v	Generates <code>select_initial</code>
18	sel_k4.v	ROM vs. external input MUX
19	k1_rom4.v	Training ROM for k_1
20	k2_rom4.v	Training ROM for k_2
21	t1_rom4.v	Training ROM for target t_1
22	nn_core4_tb.v	Testbench (sim only, +SIMULATION flag)

Chapter B

CDL File Summary

Table B.1: CDL Files Generated During the Internship

File	Lines	Contents
cell_inv.cdl	8	INVX1 transistor netlist
cell_nand2.cdl	10	NAND2X1 transistor netlist
cell_nand4.cdl	14	NAND4 transistor netlist
cell_tg.cdl	8	Transmission gate
cell_dlat.cdl	18	D-latch
cell_rom_array.cdl	45	NOR ROM bitcell array
mem4_slim.cdl	85	Complete mem4 hierarchy
forward4.cdl	1,276	forward4 + all sub-modules
backward4.cdl	2,241	backward4 + all sub-modules
nn_core4.cdl	466	Top-level chip
Total	4,171	

Chapter C

Internship Photos



Figure C.1: With faculty mentor Prof. Tapas Kumar Maiti at DA-IICT, Gandhinagar. Photo taken at the DA-IICT VLSI laboratory, June 2026.



Figure C.2: Working at the Cadence EDA workstation in the DA-IICT VLSI Lab, showing the Cadence Virtuoso Layout Editor with the `nn_core4` physical layout.